

Certifying Model Upgrades with Slice-Wise Non-Regression and Incumbent Fallback

Shengwei Zhang¹ Tao Wu² Fei Qian²

¹University of Pennsylvania

²Alibaba International Digital Commerce

August 28, 2026

Abstract

An updated model can improve an aggregate metric while degrading a slice that matters to a downstream user. We study checkpoint selection subject to non-regression tolerances relative to a retained incumbent. The central distinction is between failing to detect harm and certifying non-inferiority: the former can release harmful updates with high probability when evaluation is noisy. We give a reproducible release procedure that separates candidate search from independent, paired evaluation and returns the exact incumbent when certification fails. Applying established intersection–union and Learn-then-Test principles, we state finite-sample guarantees for one frozen candidate, a finite candidate library, and a prespecified testing order. A joint release decision does not require a slice-count Bonferroni penalty, although certification power can still decrease with the number of slices. In bounded-score simulations, a no-detected-harm gate releases a harmful candidate in 99.7% of trials in one 32-slice setting, compared with 2.6% for an exact non-inferiority gate at a 5% target. A constructed two-block family yields larger certified utility than a scalar path under matched candidate counts. Public digits experiments, including a subsequent continuation that improves average aggregate accuracy, return the incumbent in every run because certification is underpowered. These results establish an auditable protocol and its limitations; they do not establish benefits on foundation-model or multilingual translation upgrades.

1 Introduction

Model quality is often summarized by an average over tasks, classes, languages, or customer populations. An average increase permits a decrease on any one of these slices. This matters when a downstream application depends on a particular capability of the deployed model, rather than on the aggregate used to choose its successor. Version compatibility is already an established problem: [Echterhoff et al. \(2024\)](#), for example, document instance regressions when language-model bases and their downstream adapters are updated. Slice-level expectations capture a different requirement, one that can coexist with instance-level compatibility metrics.

We consider an incumbent model θ_0 , an updated checkpoint θ_1 , and a set of predefined slices. The objective is to choose a useful update while bounding the probability of releasing a model whose expected score falls more than a specified tolerance on any slice. A retained incumbent makes abstention operational: when the evidence is insufficient, the procedure can return the exact existing model. Weight interpolation supplies one possible family of intermediate candidates, and block-specific interpolation can enlarge that family. Neither the incumbent anchor nor interpolation is new; their usefulness depends on how candidate selection and statistical certification are combined.

Two errors can obscure this decision. First, a rule that releases a model whenever a regression is not statistically significant answers a different question from a non-inferiority test. More uncertainty makes the former rule easier to pass. Second, a confidence statement for one prespecified model can fail after searching over models on the same evaluation data. A final sweep over every slice does not repair this candidate-selection problem if the sweep itself is repeatedly reused.

We address these issues through an explicit interface between development and release. All candidate generation, layer search, tolerance setting, and prioritization use development data. Certification then evaluates a frozen candidate or library on independent paired observations. A failure ends the attempt and invokes the incumbent fallback. This interface can also certify candidates produced by continued training, replay, distillation, or multi-objective merging; these approaches are compatible with the release protocol.

Our contribution is an operational formulation and a reproducible examination of this interface. We (i) distinguish release safety from harm detection, including their different multiplicity requirements; (ii) specialize existing statistical tools to paired, incumbent-relative constraints with explicit fallback and search assumptions; and (iii) run controlled simulations and a small public-data integration study that report both accepted utility and failed certification. The statistical proofs apply classical concentration, intersection–union testing, and Learn-then-Test reasoning. We do not claim a new general testing principle, a new merging operator, or empirical superiority on large models.

2 Problem and scope

Let $\mathcal{S} = \{1, \dots, K\}$ denote slices fixed before certification. Slice s has distribution P_s and a measurable, higher-is-better score $h_s(\theta, Z)$. Define

$$q_s(\theta) = \mathbb{E}_{Z \sim P_s}[h_s(\theta, Z)], \quad D_s(\theta) = q_s(\theta) - q_s(\theta_0). \quad (1)$$

The tolerances $\varepsilon_s \geq 0$ are selected before viewing certification results. The population feasible set is

$$\mathcal{F} = \{\theta : D_s(\theta) \geq -\varepsilon_s \text{ for every } s\}. \quad (2)$$

For interpretable, predeclared weights $w_s \geq 0$, $\sum_s w_s = 1$, a possible utility is $U(\theta) = \sum_s w_s D_s(\theta)$. Different slice metrics are permitted in the constraints, but their aggregate needs an explicit utility conversion. Our experiments use a common score scale.

We seek a release rule $\hat{\theta} = \mathcal{A}(\mathcal{D}_{\text{dev}}, \mathcal{D}_{\text{cert}})$ such that

$$\mathbb{P} \left\{ \exists s : D_s(\hat{\theta}) < -\varepsilon_s \right\} \leq \alpha. \quad (3)$$

The probability is over the data and any declared algorithmic randomization. This is an unconditional repeated-sampling guarantee for the release procedure, not a posterior probability for the observed model, and not a bound conditional on an upgrade being released. Its scope is the specified scores, slices, and evaluation distributions. It does not guarantee individual predictions, undiscovered slices, or future distribution shifts.

Paired evaluation and the anchor. On certification examples Z_{is} , compare models using

$$X_{is}(\theta) = h_s(\theta, Z_{is}) - h_s(\theta_0, Z_{is}), \quad \hat{D}_s(\theta) = \frac{1}{n_s} \sum_{i=1}^{n_s} X_{is}(\theta). \quad (4)$$

At $\theta = \theta_0$, $X_{is}(\theta_0) = 0$ by identity, so the incumbent is known to satisfy (2); it is not estimated to be safe from a noisy self-comparison. For stochastic generation, reuse the incumbent artifact and its evaluation outputs, or otherwise make the identity comparison explicit. An independently resampled generation from the same weights need not have zero realized score difference.

Assumptions for finite-sample certification. Conditioning on development data, each candidate and score function is fixed; certification examples within each slice are independent and identically distributed from P_s ; and each paired difference lies in a known interval $[a_s, b_s]$ of width $W_s = b_s - a_s$. Dependence across slices is allowed. For $h_s \in [0, 1]$, the generic difference range is $[-1, 1]$, so $W_s = 2$. Tighter structural bounds may be used if justified before seeing certification data. An observed minimum and maximum are not automatically valid population bounds.

Checkpoint compatibility. A block interpolation family takes the form

$$\theta_b(\lambda) = (1 - \lambda_b)\theta_{0,b} + \lambda_b\theta_{1,b}, \quad \lambda \in [0, 1]^B. \quad (5)$$

It requires compatible parameter names, shapes, and semantics. Tokenizers and token indices must also agree when they determine embedding meaning. Equal architecture alone does not establish alignment; permutation symmetries can obstruct averaging (Ainsworth et al., 2023). Common checkpoint ancestry is a practical starting point, not a guarantee that every interpolated model performs well. No certification proof below assumes a smooth or monotone interpolation path.

3 From candidate search to a defensible release

3.1 Three decisions that must be distinguished

For a scalar comparison with standard error σ and a positive normal quantile z , the following decisions differ:

$$\text{Point estimate: } \hat{D} \geq -\varepsilon; \quad (6)$$

$$\text{No detected harm: } \hat{D} + z\sigma \geq -\varepsilon; \quad (7)$$

$$\text{Non-inferiority certificate: } \hat{D} - z\sigma \geq -\varepsilon. \quad (8)$$

Equation (7) rules out updates only when an upper confidence bound lies below the tolerance. It can control false alarms about safe models. Equation (8) requires a lower bound to clear the tolerance and can control falsely certifying unsafe models. Failure to reject inferiority does not establish non-inferiority.

For illustration, assume an exactly normal estimator with $D = -0.20$, $\varepsilon = 0.05$, $\sigma = 0.10$, and $z = \Phi^{-1}(0.95)$. The no-detected-harm rule releases with probability about 0.558, while the non-inferiority rule releases with probability about 0.00083. This is a calculation in the specified normal model, not an assumption about arbitrary evaluation metrics. Increasing the quantile to obtain simultaneous harm detection makes (7) easier to pass, which can increase release risk.

3.2 One frozen candidate: an intersection–union test

For each slice, test the null $H_s : D_s(\theta) \leq -\varepsilon_s$ with a valid one-sided p -value p_s . Validity means $\mathbb{P}(p_s \leq t) \leq t$ under H_s , for every $t \in [0, 1]$. The unsafe null for the whole candidate is a union, $H = \bigcup_s H_s$. Accordingly, define

$$p(\theta) = \max_s p_s(\theta), \quad \text{certify } \theta \text{ only if } p(\theta) \leq \alpha. \quad (9)$$

The use of a closed null at the tolerance boundary is conservative for (3), which counts strict violations.

Theorem 1 (Frozen-candidate release; standard IUT application). *Suppose development data select one candidate independently of the certification observations. If the slice p -values are conditionally valid given development data, then releasing the candidate only when (9) holds, and otherwise returning θ_0 , satisfies (3). No independence across slices and no division of α by K are required.*

Proof. Condition on development data. If the selected candidate is strictly unsafe, fix one violating slice s^* . Its null is true, and the event of release is contained in $\{p_{s^*} \leq \alpha\}$, whose probability is at most α . A feasible candidate or the exact incumbent cannot cause the event in (3). Integrate over development data. $\square$

This familiar intersection–union distinction also appears in multi-endpoint testing: establishing a conjunction of endpoint requirements differs from selecting one successful endpoint (U.S. Food and Drug Administration, 2022). Theorem 1 concerns a joint decision. It does not assert simultaneous numerical coverage of every marginal confidence bound. If an application needs all K intervals to cover jointly, an appropriate simultaneous procedure is still needed.

A bounded-score implementation. Hoeffding’s inequality (Hoeffding, 1963) gives

$$L_s(\eta) = \hat{D}_s - W_s \sqrt{\frac{\log(1/\eta)}{2n_s}}. \quad (10)$$

Testing all $L_s(\alpha) \geq -\varepsilon_s$ implements Theorem 1. For general scores in $[0, 1]$, the radius is $\sqrt{2\log(1/\alpha)/n_s}$. Exact tests for special score distributions, or other valid bounds, can replace Hoeffding. A plug-in normal or bootstrap interval is not automatically a finite-sample certificate.

3.3 Several candidates: multiplicity belongs to the search

Let development data freeze J candidates $\theta^{(1)}, \dots, \theta^{(J)}$. The same certification observations may evaluate all of them; resulting p -values can be dependent. Testing each at level α and selecting any passing candidate does not generally preserve (3).

Theorem 2 (Finite library and fixed order; LTT applications). *Under the conditional validity assumption of Theorem 1, either procedure below satisfies (3).*

1. *Assign nonnegative levels η_j using development data, with $\sum_j \eta_j \leq \alpha$. Certify candidate j when $p(\theta^{(j)}) \leq \eta_j$; select any certified candidate or return θ_0 .*
2. *Freeze an order of candidates using development data. Test in that order at level α , stop at the first failure, and select among candidates certified before that failure, or return θ_0 .*

Proof. For the first procedure, each unsafe candidate is falsely certified with probability at most η_j . A union bound over unsafe candidates gives at most $\sum_j \eta_j \leq \alpha$. For the second, condition on development data and identify the first candidate in the frozen order whose whole-candidate null is true. Certifying any candidate with a true null requires rejecting that first true null, an event of probability at most α . If no candidate has a true null, every candidate is strictly feasible. The incumbent fallback is exact. $\square$

The equal allocation is $\eta_j = \alpha/J$, with no additional K factor for the joint decision. Fixed-sequence validity does not require population feasibility to be monotone in the order; a poor order can instead reduce power by causing early termination. These are instances of the Learn-then-Test framework (Angelopoulos et al., 2025), not new multiple-testing results.

Stage	Required operation
1	Retain the exact incumbent. Before certification, define slices, scores, tolerances, utility, sampling units, and error budget.
2	On development data, generate and score compatible candidates. Freeze one candidate, or freeze a library and its valid testing plan.
3	Evaluate incumbent and frozen candidate(s) on the same certification examples; preserve paired observations and any cross-slice clustering.
4	Apply the selected whole-candidate non-inferiority test. Release only an eligible candidate. Otherwise return the incumbent and end this attempt.
5	Record the selected artifact, gate, sample sizes, and failure outcome. Use separate reporting data for descriptive performance analysis.

Table 1: Candidate generation and certification are separate operations. Repair after a failed gate starts a new statistical attempt and requires new data or a predeclared valid sequential design.

3.4 Development search and failure handling

Our simplest deployment interface freezes one candidate. Development search may evaluate a scalar grid, a block grid, coordinate proposals, or a preexisting merging method. It estimates utility and anticipates the eventual certification margin, then hands a single immutable candidate to the release test. Table 1 states the protocol. More expensive libraries use Theorem 2.

Active-slice screening is allowed during development. A slice initially far from its threshold can become binding after a block update, so the release gate still checks every predefined slice. If that gate fails, detecting the failure is not the same as finding another acceptable model. We explicitly return the incumbent rather than repeatedly adjusting the candidate on certification data.

For a library of J checkpoints, naive development evaluation costs J full slice sweeps, with additional model-loading and generation costs. A screened block search may reduce some development measurements, but its initial sweep, number of blocks, proposal rounds, and final certification still count. We do not assert that evaluation is always cheaper than continued training. Candidate generators should be compared at matched evaluation and training budgets, then passed through the same gate.

4 What the guarantee costs and what the search can gain

4.1 Slice count affects power without a slice-count testing penalty

Let a frozen candidate have positive margins $\gamma_s = D_s + \varepsilon_s > 0$. For the Hoeffding implementation, denote the radius in (10) at level α by r_s . If $\gamma_s > r_s$, then

$$\mathbb{P}\{L_s(\alpha) < -\varepsilon_s\} \leq \exp \left[-\frac{2n_s(\gamma_s - r_s)^2}{W_s^2} \right]. \quad (11)$$

Thus, a sufficient condition for certifying the candidate with probability at least $1 - \beta$ is

$$n_s \geq \frac{W_s^2}{2\gamma_s^2} \left(\sqrt{\log(1/\alpha)} + \sqrt{\log(K/\beta)} \right)^2 \quad \text{for every } s. \quad (12)$$

The proof is in Appendix A. Equation (12) separates safety level α from joint power $1 - \beta$. It does not promise acceptance uniformly in K . A slice close to its tolerance can require substantial data, and exact boundary certification may have low power even with a correct implementation.

Paired measurements offer a principled way to improve precision:

$$\text{Var}(h_s(\theta, Z) - h_s(\theta_0, Z)) = \text{Var}(h_s(\theta, Z)) + \text{Var}(h_s(\theta_0, Z)) - 2 \text{Cov}(h_s(\theta, Z), h_s(\theta_0, Z)). \quad (13)$$

Positive covariance reduces the variance of the difference relative to independent evaluations. It does not justify changing the direction of a confidence bound. Range-based Hoeffding bounds do not exploit every variance reduction; variance-sensitive valid tests are a useful extension.

4.2 Scalar paths and block freedom

The identity $\theta(0) = \theta_0$ ensures that the population feasible set contains the incumbent. A scalar interpolation only explores the diagonal $\lambda_1 = \dots = \lambda_B$, whereas a block search includes off-diagonal candidates. An enlarged family cannot reduce the best population utility if the old family is included, but a finite development search and certification penalty can eliminate this advantage in practice.

A scalar “first binding threshold” is globally optimal only under stronger geometry assumptions. If each slice’s entire feasible coefficient set equals $[0, t_s]$, and utility is nondecreasing over their intersection, then the scalar optimum is $\min_s t_s$. If a slice has disconnected feasible regions, this only describes the component attached to zero. Evaluating a finite grid and selecting its best certified member requires no connectedness assumption, although it does not certify coefficients between evaluated grid points.

Our controlled geometry has eight slices and two blocks:

$$D_1(\lambda) = -0.08\lambda_1 + 0.02\lambda_2, \quad D_s(\lambda) = 0.10\lambda_1 + 0.04\lambda_2 \quad (s \geq 2), \quad (14)$$

with $\varepsilon_s = 0.01$ and uniform weights. The scalar feasible endpoint is $\lambda_1 = \lambda_2 = 1/6$, giving utility 0.01917. The continuous block optimum is $(0.375, 1)$, giving 0.06656. This explicitly constructed separation motivates a test of whether the search advantage survives finite-data certification. It is not evidence that fragile capabilities in natural networks are localized to particular layers.

5 Experiments

5.1 Design, provenance, and reporting

All numerical results in this section were executed locally from the accompanying code. The synthetic studies use explicit bounded paired-score distributions, seed 20260911, independent random streams, 20000 independent trials for gate studies, and 4000 trials per candidate-search configuration. Counts sampled directly from binomial distributions are exact sufficient statistics for the described observations, not a Gaussian approximation. Saved artifacts include these counts, trial-level decisions, selected coefficients, CSV summaries, generated figures, and a source-hash manifest. Rates have exact 95% Clopper–Pearson intervals; utility intervals quantify Monte Carlo uncertainty. Main figures show representative settings; full sample-size and correlation results are retained in machine-readable tables.

The primary outcomes are the probability of a harmful release, probability of a non-incumbent release, and population utility of the deployed model. Synthetic population expectations are known by construction. The retained-gain ratio is reported only in the geometry experiment, whose candidate gain is exactly positive (0.115). Real-data reporting uses absolute accuracy differences and does not divide by a negative candidate gain.

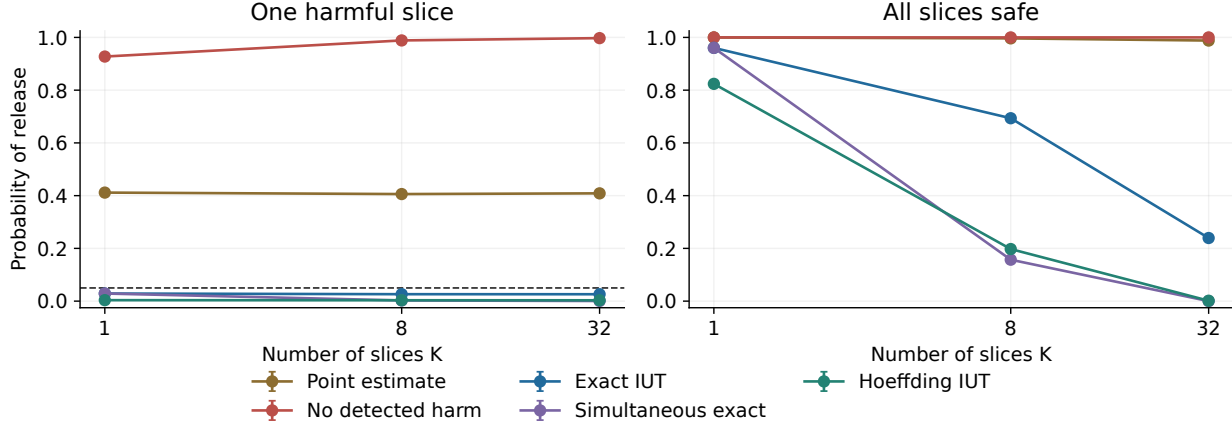


Figure 1: Release probability at $n = 512$ examples per slice. Left: release is an error because one slice is beyond tolerance. Right: every slice is safe, so release measures power. Error bars are exact 95% Monte Carlo intervals, often smaller than the markers. The dashed line marks the 5% false-release target. A simultaneous harm-detection screen increasingly accepts the harmful candidate as K grows; this behavior is not a non-inferiority guarantee.

Rule	$K = 1$	$K = 8$	$K = 32$
Point estimate	41.16	40.60	40.87
No detected harm	92.74	98.86	99.73
Exact IUT	2.89	2.63	2.61
Simultaneous exact	2.89	0.26	0.05
Hoeffding IUT	0.36	0.33	0.32

Table 2: Harmful-release rates (%) over 20000 trials at $n = 512$. The true violation is 0.002 beyond the tolerance. All methods see identical draws in each configuration.

5.2 Harm detection and certification have different operating behavior

Let $X_s \in \{-0.2, +0.2\}$ with mean D_s , $\varepsilon_s = 0.02$, and $n \in \{128, 512, 2048\}$ observations per slice. In the harmful scenario, one slice has $D_1 = -0.022$ and the others have $D_s = 0.08$. In the safe scenario, every slice has $D_s = 0.01$. We vary $K \in \{1, 8, 32\}$.

The exact endpoint non-inferiority test uses $C_s \sim \text{Binomial}(n, p_s)$, $p_0 = (0.2 - 0.02)/0.4 = 0.45$, and the upper-tail p -value $\mathbb{P}_{p_0}(C \geq C_s)$. Exact IUT requires all these p -values to be at most α . The simultaneous exact comparator uses α/K ; it represents a more demanding marginal familywise construction. The no-detected-harm gate instead rejects a model when at least one lower-tail harm test is significant at α/K . We also evaluate the point-estimate and bounded Hoeffding IUT gates.

Table 2 and Figure 1 show the difference. At $K = 32, n = 512$, the no-detected-harm rule releases the harmful candidate in 99.7% of trials, while exact IUT releases it in 2.6%. The latter is below the 5% target, with discreteness and the nonzero violation making it conservative. The point-estimate rule has no comparable protection. The simultaneous exact and Hoeffding gates are more conservative in this family. In the safe scenario, requiring all slices to pass reduces power as K increases; Theorem 1 removes a testing penalty, not this conjunction effect.

We also repeat the 32-slice study with 0%, 50%, or 100% of example positions sharing a common latent uniform variable across slices. Thresholding this uniform at each slice’s success probability

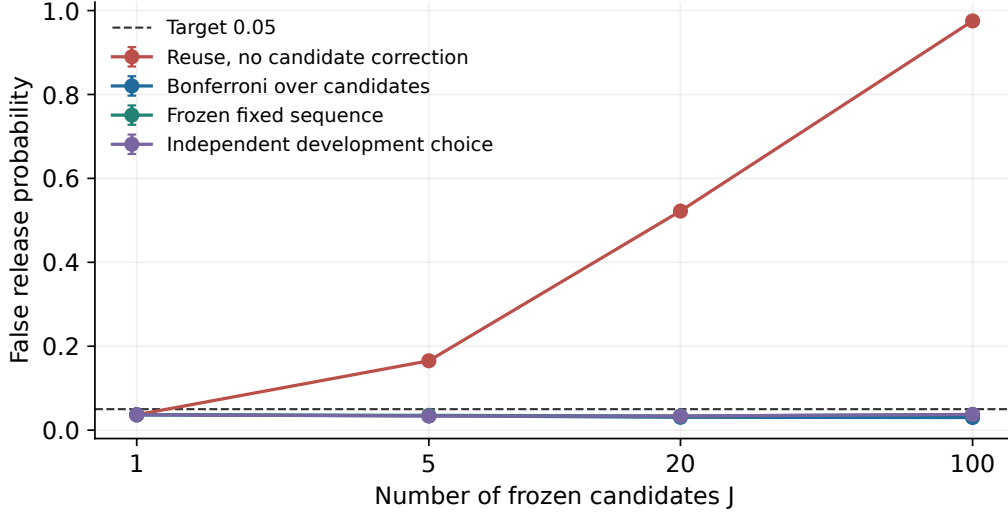


Figure 2: False-release probability after searching a frozen library of harmful candidates. At $J = 100$, uncorrected reuse releases harm in 97.5% of trials; candidate correction gives 3.0%. Fixed-sequence testing and independent development choice preserve their designated error target. Candidate independence is a simulation setting, not a requirement of the validity proofs.

preserves each binomial marginal while introducing dependence. The exact IUT harmful-release frequency remains below 5% for each tested dependence fraction. The theorem, rather than this finite sweep, establishes validity under arbitrary cross-slice dependence satisfying the within-slice assumptions.

5.3 Searching certification results inflates false release

We freeze $J \in \{1, 5, 20, 100\}$ synthetic candidates, each with a score mean $D = -0.0205$, tolerance 0.02, and $n = 2048$ observations. Candidate-specific score observations are independent in this stress test. Every non-incumbent release is therefore harmful. We compare selecting any candidate that passes an uncorrected level-0.05 test, candidate Bonferroni correction, a frozen fixed sequence, and choosing one candidate on an independent development split before certification.

Figure 2 shows that an individually valid test does not remain valid when any success in a large search can trigger deployment. At $J = 100$, the uncorrected rule reaches 97.5% harmful releases, compared with 3.0% using α/J . The all-harmful library makes the fixed-sequence interpretation particularly transparent: any false certification requires the first candidate to pass. Independent development selection permits searching the development data freely, while the subsequent single test retains its validity. This study isolates statistical selection effects; it does not model the natural dependence between nearby neural checkpoints.

5.4 Block search can retain more utility in a constructed family

For the means in (14), observations are

$$X_{is}(\lambda) = \mu_s^\top \lambda + 0.06(\lambda_1 V_{is1} + \lambda_2 V_{is2}), \quad V_{isb} \stackrel{\text{iid}}{\sim} \text{Rademacher}. \quad (15)$$

These are valid paired differences between scores in $[0, 1]$: an incumbent score in $[0.48, 0.52]$ can share its base noise with the updated score. Each observed coefficient lies within $[-0.2, 0.2]$, which

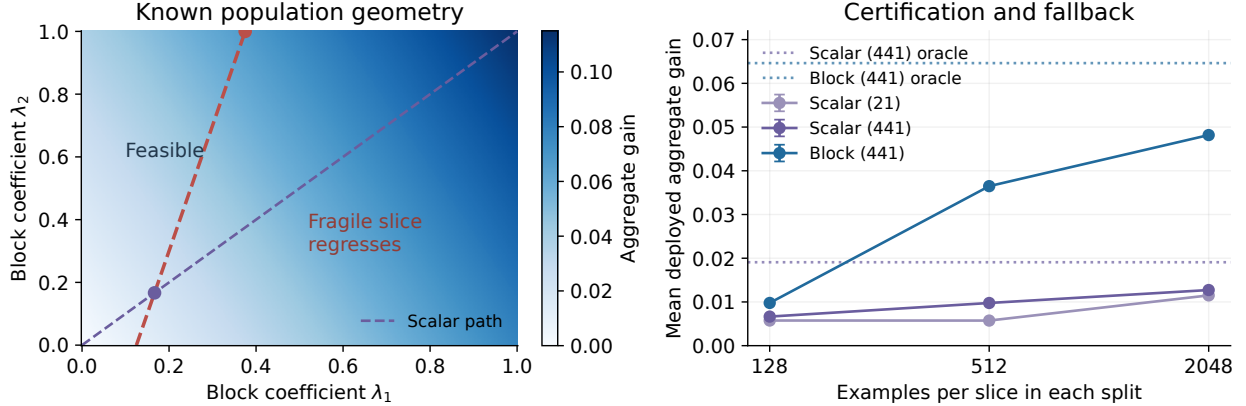


Figure 3: Left: the population feasible region for the constructed two-block family, with continuous scalar and block optima marked. Right: mean deployed utility after independent certification and incumbent fallback; dotted lines give the corresponding finite-grid population optima. All search families use the same development and certification draws. Error bars show 95% Monte Carlo mean intervals.

n	Family	Gain	Retained (%)	Fallback (%)	Harmful releases
128	Scalar (21)	0.0058	5.0	0.00	0/4000
128	Scalar (441)	0.0066	5.8	4.83	0/4000
128	Block (441)	0.0098	8.5	4.83	0/4000
512	Scalar (21)	0.0057	5.0	0.22	0/4000
512	Scalar (441)	0.0098	8.5	4.70	0/4000
512	Block (441)	0.0365	31.7	3.52	0/4000
2048	Scalar (21)	0.0115	10.0	0.00	0/4000
2048	Scalar (441)	0.0127	11.1	4.45	0/4000
2048	Block (441)	0.0482	41.9	2.50	0/4000

Table 3: Utility after certification. Parentheses indicate candidate counts. Retained gain is relative to the known full-update gain of 0.115. Fallback is an actual output of the procedure, not an excluded trial.

provides the declared difference width $W(\lambda) = 0.4\|\lambda\|_1$. The identity candidate has width zero.

Each trial generates independent development and certification splits of n examples per slice. Search maximizes development utility among candidates satisfying a development margin that anticipates the Hoeffding certification radius and a fixed design buffer (Appendix B). It then freezes one candidate and certifies it once. We compare a coarse 21-point scalar grid, a 441-point scalar grid, and a 21×21 block grid with 441 candidates. The dense scalar and block grids match candidate counts; the coarse grid is a resolution diagnostic. The block family has a known geometric advantage by construction, and this comparison is not a general claim about equal wall-clock search costs.

At $n = 2048$, matched-count block search achieves mean utility 0.0482, compared with 0.0127 for dense scalar search, a factor of 3.79 in this constructed family. Certification failures reduce the gains actually released, and coarse scalar resolution creates additional effects at intermediate sample sizes. No harmful release occurs in any of the nine search configurations. For each configuration’s 4000 trials, zero events correspond to a two-sided 95% upper confidence endpoint of 0.092%, not proof of zero risk. The theorem supplies the risk bound; the experiment validates the implemented behavior under this generator.

5.5 A public digits integration test exposes the data bottleneck

We additionally train and merge actual small neural checkpoints on the public optical digits data distributed by scikit-learn (scikit-learn developers, 2026; Alpaydin and Kaynak, 1998). Across five fixed seeds, a 64–48–10 ReLU network is trained for 12 epochs; the update continues for 24 epochs using only classes 0–4. Both scalar and layer searches evaluate 121 development candidates. Data are split into training, development, certification, and reporting partitions before fitting. The continuation is deliberately a forgetting stress test, and its aggregate accuracy also deteriorates; it is not an aggregate-improving upgrade example.

Mean reporting accuracies are 0.9478 for the incumbent, 0.7300 for the updated checkpoint, 0.9494 for the scalar candidate, and 0.9483 for the layer candidate. The protocol freezes its proposed candidate using development data. Every proposal fails the bounded Hoeffding release gate: certification has only 35–37 observations per class and tolerance 0.03. The actual released model is the incumbent in all five runs. This result validates end-to-end fallback and illustrates severe power limits; it does not establish a useful certified upgrade, layer superiority, or a population guarantee beyond the sampling assumptions. Full configurations, descriptive seed variation, weights, split indices, and predictions are included in Appendix C and the artifact.

After reviewing this stress test, we fixed and executed a second protocol that continues on all ten training classes. Across the same five seeds, the continued checkpoint reaches mean reporting accuracy 0.9594, compared with 0.9567 for scalar and 0.9544 for layer proposals; all frozen proposals still fail the gate. Thus a path with higher average aggregate accuracy also encounters the certification bottleneck, while merging has no mean accuracy advantage over its fully continued endpoint here. This subsequent extension reuses the original splits and is descriptive, not fresh confirmatory certification. Its unchanged 24 epochs involve more training updates than the selective path, so the comparison does not isolate class coverage at matched compute.

6 Relation to existing work

Merging and multi-objective selection. WiSE-FT interpolates pretrained and fine-tuned weights to improve robustness (Wortsman et al., 2022b); model soups study averaging related fine-tuned models (Wortsman et al., 2022a). We use this family of operations rather than claim a new one. MAP explicitly estimates Pareto fronts of merge coefficients, and Pareto Merging learns preference-conditioned tradeoffs (Li et al., 2025; Chen and Kwok, 2025). It would therefore be incorrect to characterize all existing merging methods as optimizing only one fixed average. A relative slice constraint defines an admissible portion of a tradeoff set; our focus is how to certify a selected candidate with finite paired data. A strong empirical comparison would pass candidates from these methods through the same release interface.

Statistical and safe-update frameworks. Learn then Test provides the direct precedent for calibrating a parameterized predictor through multiple testing (Angelopoulos et al., 2025). Seldonian algorithms separate candidate selection from safety testing and can return no solution (Hoag et al., 2023). We instantiate these ideas with an incumbent-relative difference and an exact operational fallback. SafeOpt studies optimization from safe regions under Gaussian-process regularity, including safety of queried points (Sui et al., 2015); our offline evaluation points need not themselves be safe to deploy. Elmecker-Plakolm et al. (2026) compute locally invariant parameter domains for safe updates, explicitly include the no-op anchor, and provide finite-sample concentration arguments. Our certificates apply to frozen evaluated candidates rather than certify a connected parameter

domain. These are differences in the certified object and workflow, not a claim that prior safe-update work lacks statistical guarantees.

Update compatibility and evaluation. MUSCLE addresses LLM update compatibility through instance regressions and adapters (Echterhoff et al., 2024). A slice can maintain its mean while some formerly correct examples become wrong, so slice non-regression does not subsume negative-flip protection. Paired significance testing in NLP has a long history (Koehn, 2004; Dror et al., 2018); the relevant additional issue here is the use of evaluation results to make a release decision after candidate search.

7 Limitations and conclusions

The numerical evidence is mainly synthetic. It demonstrates that a block family can contain useful candidates and that different release gates have different error behavior under known distributions. It does not show that this geometric advantage is common, that the proposed search is computationally superior, or that language-model upgrades can be certified with realistic evaluation budgets. The public digits studies are small and yield no certified non-incumbent upgrade; the subsequent balanced path also uses previously examined holdouts and supplies descriptive evidence only.

The bounded-score theorem assumes representative independent examples within each slice. Shared documents, writers, or annotation batches can violate this assumption and require an appropriate cluster-level design. Unbounded metrics and corpus-level scores need their own statistical treatment. Repeated organizational releases consume additional error and tolerance budgets; the one-attempt guarantee here does not certify an unrestricted sequence of updates. Positive tolerances allow bounded degradation, and utility improvement requires an additional condition if it must itself be certified.

The actionable conclusion is a specific release discipline: generate candidates on development data, preserve paired incumbent comparisons, use non-inferiority in the correct direction, account for selection on certification results, and return the exact incumbent when evidence is inadequate. Low-dimensional merging can make that discipline convenient, but cannot substitute for adequate data or correct inference. The accompanying proofs and experiments make both successful selection and fallback reviewable.

Reproducibility statement. The artifact contains executable experiment scripts, independent gate tests, raw sufficient statistics, neural checkpoints, split indices, result tables, figure sources, and environment/source manifests. All numerical tables in this manuscript are generated from completed runs. No private corpus, internal deployment statistic, or unrun large-model result is included.

AI assistance statement. AI tools assisted with critique of the initial proposal, repair and checking of mathematical derivations, literature searches, experimental design and implementation, execution and inspection of local runs, and manuscript drafting. The recorded results come from executed programs. The authors are responsible for all claims, derivations, sources, code, and results, including the validation of AI-assisted content.

References

- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=CQsmMYm1P5T>.
- E. Alpaydin and C. Kaynak. Optical recognition of handwritten digits, 1998. URL <https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits>.
- Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. Learn then test: Calibrating predictive algorithms to achieve risk control. *The Annals of Applied Statistics*, 19(2): 1641–1662, 2025. doi: 10.1214/24-AOAS1998. URL <https://doi.org/10.1214/24-AOAS1998>.
- Weiyu Chen and James Kwok. Pareto merging: Multi-objective optimization for preference-aware model merging. In *International Conference on Machine Learning*, volume 267, pages 8345–8363, 2025. URL <https://proceedings.mlr.press/v267/chen25af.html>.
- Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker’s guide to testing statistical significance in natural language processing. In *Annual Meeting of the Association for Computational Linguistics*, pages 1383–1392, 2018. doi: 10.18653/v1/P18-1128. URL <https://aclanthology.org/P18-1128/>.
- Jessica Maria Echterhoff, Fartash Faghri, Raviteja Vemulapalli, Ting-Yao Hu, Chun-Liang Li, Oncel Tuzel, and Hadi Pouransari. MUSCLE: A model update strategy for compatible LLM evolution. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7320–7332, 2024. doi: 10.18653/v1/2024.findings-emnlp.430. URL <https://aclanthology.org/2024.findings-emnlp.430/>.
- Leo Elmecker-Plakolm, Pierre Fasterling, Philip Sosnin, Calvin Tsay, and Matthew Wicker. Provably safe model updates. In *IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. URL <https://arxiv.org/abs/2512.01899>. arXiv:2512.01899.
- Austin Hoag, James E. Kostas, Bruno Castro da Silva, Philip S. Thomas, and Yuriy Brun. Seldonian toolkit: Building software with safe and fair machine learning. In *IEEE/ACM International Conference on Software Engineering: Companion Proceedings*, pages 107–111, 2023. doi: 10.1109/ICSE-Companion58688.2023.00035. URL https://people.cs.umass.edu/~bsilva/papers/seldonian_toolkit_ICSE_2023.pdf.
- Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963. doi: 10.1080/01621459.1963.10500830. URL <https://doi.org/10.1080/01621459.1963.10500830>.
- Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Conference on Empirical Methods in Natural Language Processing*, pages 388–395, 2004. URL <https://aclanthology.org/W04-3250/>.
- Lu Li, Tianyu Zhang, Zhiqi Bu, Suyuchen Wang, Huan He, Jie Fu, Jiang Bian, Yonghui Wu, Yong Chen, and Yoshua Bengio. MAP: Low-compute model merging with amortized pareto fronts via quadratic approximation. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a3724221b623bf1966e92cd266ac6177-Abstract-Conference.html.
- scikit-learn developers. sklearn.datasets.load_digits, 2026. URL https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html. Documentation accessed September 11, 2026.
- Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. Safe exploration for optimization with gaussian processes. In *International Conference on Machine Learning*, volume 37, pages 997–1005, 2015. URL <https://proceedings.mlr.press/v37/sui15.html>.
- U.S. Food and Drug Administration. Multiple endpoints in clinical trials: Guidance for industry, 2022. URL <https://www.fda.gov/media/162416/download>.
- Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S.

- Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, volume 162, pages 23965–23998, 2022a. URL <https://proceedings.mlr.press/v162/wortsman22a.html>.
- Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7959–7971, 2022b. URL https://openaccess.thecvf.com/content/CVPR2022/html/Wortsman_Robust_Fine-Tuning_of_Zero-Shot_Models_CVPR_2022_paper.html.

A Proofs, boundary cases, and interpretation

A.1 Validity of the bounded paired test

For a frozen candidate, assume $X_{is} \in [a_s, b_s]$ are iid with mean D_s . Hoeffding gives, for every $t > 0$,

$$\mathbb{P}\{\hat{D}_s - D_s \geq t\} \leq \exp(-2n_s t^2 / W_s^2). \quad (16)$$

Under $D_s \leq -\varepsilon_s$, the event $\hat{D}_s - W_s \sqrt{\log(1/\eta)/(2n_s)} \geq -\varepsilon_s$ implies an upward deviation of at least the radius. Its probability is at most η . Equivalently, a valid conservative p -value for $W_s > 0$ is

$$p_s = \exp \left[-\frac{2n_s \{\max(\hat{D}_s + \varepsilon_s, 0)\}^2}{W_s^2} \right]. \quad (17)$$

At the incumbent, $W_s = 0$ and the difference is deterministically zero; this identity is handled separately instead of dividing by zero or constructing a random boundary test. A zero empirical variance for a distinct model does not give this identity guarantee.

Conditional validity is essential. Development data may determine model coefficients, slice weights, tolerances, or test order before certification, provided the resulting testing problem satisfies the stated assumptions conditional on that information. If certification outcomes alter those quantities, the above fixed-candidate calculation is no longer sufficient. An independent new sample or a valid sequential construction is then needed.

A.2 Power calculation

The failure event for a truly interior candidate is

$$L_s(\alpha) < -\varepsilon_s \iff \hat{D}_s - D_s < -(\gamma_s - r_s).$$

When $\gamma_s > r_s$, the lower-tail Hoeffding bound gives (11). To make this probability at most β/K , it suffices that

$$\gamma_s \geq \frac{W_s}{\sqrt{2n_s}} \left(\sqrt{\log(1/\alpha)} + \sqrt{\log(K/\beta)} \right).$$

Squaring and rearranging gives (12); union-bounding the K failure events yields joint acceptance probability at least $1 - \beta$. For equal candidate allocation replace α inside the radius by α/J . The result is sufficient, potentially conservative, and does not claim necessity or optimal sample complexity.

A.3 When point-estimate acceptance can decay exponentially

An illustrative Gaussian calculation can explain a many-slice power problem, but requires its assumptions to be stated correctly. Fix one candidate and assume independent errors $\hat{D}_s = D_s + \sigma_s Z_s$, $Z_s \sim N(0, 1)$. Suppose at least m slices have bounded standardized slack,

$$0 \leq (D_s + \varepsilon_s)/\sigma_s \leq B < \infty. \quad (18)$$

Then

$$\mathbb{P}\{\text{all point-estimate constraints pass}\} = \prod_s \Phi((D_s + \varepsilon_s)/\sigma_s) \leq \Phi(B)^m. \quad (19)$$

If $m \geq \rho K$, the bound decays exponentially in K . A lower bound on slack and an upper bound on variance would give the opposite inequality and cannot prove this conclusion. Correlated slices,

increasing margins, or increasing sample sizes change the result. The calculation is for a fixed candidate and does not by itself establish that the coefficient selected by an arbitrary search converges to zero. In particular, an incumbent self-comparison has exactly zero paired error.

This phenomenon is a power issue, not a reason to switch to an upper-bound release screen. A valid non-inferiority procedure can also reject many truly feasible candidates when data are inadequate. The practical responses are more informative paired data, a defensible tolerance, a better candidate, or fallback.

A.4 Feasible components and repair operators

For scalar coefficients let $\Lambda_s = \{\lambda : D_s(\lambda) \geq -\varepsilon_s\}$. If $\Lambda_s = [0, t_s]$ for each slice, their intersection equals $[0, \min_s t_s]$. Monotone utility on that set gives the endpoint solution. Merely containing an anchored interval is weaker. For example, with zero tolerance,

$$D_1(\lambda) = \lambda(\lambda - 0.25)(\lambda - 0.75), \quad 0 \leq \lambda \leq 1, \quad (20)$$

the feasible set is $[0, 0.25] \cup [0.75, 1]$. Add a second slice $D_2(\lambda) = \lambda$ and choose a utility increasing sufficiently with D_2 . Stopping at the first violation misses the later feasible region. A grid can reveal sampled disconnected components; it cannot rule out unseen narrow components without further regularity assumptions.

Similarly, uncontrolled repair can cycle even with convex constraints. With $C_1 = (-\infty, 1]$, $C_2 = [-1, \infty)$, a rule that sends 2 to -2 and -2 to 2 always fixes the currently violated constraint while never entering their nonempty intersection. This shows that the repair specification is too weak, not that convexity is absent.

For completeness, in finite-dimensional Euclidean space suppose all C_s are closed and convex, their intersection F is nonempty, and a repair step is the metric projection onto the intersection of currently violated sets. Such a step is Fejér monotone relative to F :

$$\|x_{t+1} - z\|^2 \leq \|x_t - z\|^2 - \|x_{t+1} - x_t\|^2 \quad (z \in F).$$

Thus the sequence is bounded and step lengths tend to zero. Every violated set has distance from x_t at most that step length, while satisfied sets have distance zero. Any cluster point therefore lies in every closed C_s , and Fejér monotonicity implies convergence to that point. The successful operator uses additional geometry and projection control. We do not rely on either this property or failure of a repair baseline for our release guarantee. A repair pipeline can retain the same incumbent and use the same certification test.

A.5 Aggregate guarantees and repeated releases

With positive tolerances, a certified candidate can have a negative aggregate change. If aggregate non-regression is mandatory, include a separately defined aggregate difference as another endpoint and certify its lower bound as well. Appropriate mixture sampling or a valid combination of slice estimates is needed to define that endpoint; selecting the largest noisy aggregate alone does not certify improvement.

For repeated release attempts indexed by t , a predeclared allocation with $\sum_t \alpha_t \leq \alpha_{\text{total}}$ yields a union-bound control if each attempt's conditional validity assumptions hold. Tolerances also accumulate: a sequence of individually tolerated decreases can move far below the original model. One may compare every release to a fixed reference, maintain a cumulative tolerance budget, or specify a sequential target. These are distinct policies and are not implemented by the one-step experiments here.

B Synthetic generators and complete implementation details

B.1 Gate calibration

For each configuration, the script samples $C_s \sim \text{Binomial}(n, (D_s + 0.2)/0.4)$ and sets $\hat{D}_s = 0.4C_s/n - 0.2$. The same C_s are supplied to every gate. The no-detected-harm test accepts only if $\mathbb{P}_{0.45}(C \leq C_s) > \alpha/K$ for every slice; this is a lower-tail test for harm. Exact IUT instead requires $\mathbb{P}_{0.45}(C \geq C_s) \leq \alpha$ for every slice. Discrete ties are handled by these explicit inequalities. The Hoeffding width is 0.4.

For the dependence study, $n_{\text{shared}} = \rho n$ positions use shared uniforms across slices, with $\rho \in \{0, 0.5, 1\}$. In the one-harmful setting the two thresholds are $p_{\text{low}} = 0.445$ and $p_{\text{high}} = 0.70$. Counts in the shared positions come from a multinomial distribution with probabilities $(p_{\text{low}}, p_{\text{high}} - p_{\text{low}}, 1 - p_{\text{high}})$. The harmful slice uses the first cell, and each easy slice uses the first two. Remaining positions use independent binomials. In the all-safe setting all slices share the same threshold 0.525. Within each slice, all positions remain iid with the specified marginal; cross-slice dependence changes.

n	K	Harm: IUT	Harm: Hoeffding	Safe: IUT	Safe: simultaneous
128	1	3.02	0.45	48.17	48.17
128	8	3.07	0.53	0.39	0.00
128	32	3.04	0.53	0.00	0.00
512	1	2.89	0.36	96.03	96.03
512	8	2.63	0.33	69.36	15.71
512	32	2.61	0.32	23.94	0.00
2048	1	1.67	0.20	100.00	100.00
2048	8	1.65	0.19	100.00	99.99
2048	32	1.57	0.18	100.00	99.74

Table 4: Completed sample-size study: release rates (%) over 20000 trials per configuration. “Harm” is the one-harmful-slice scenario, so release is an error; “Safe” has every slice at mean 0.01, so release measures power. IUT uses exact binomial tests. The last column uses exact marginal tests at level α/K .

Shared fraction	Harm: no detection	Harm: IUT	Safe: IUT	Safe: simultaneous
0.0	99.81	2.90	24.19	0.00
0.5	99.74	2.79	62.72	8.52
1.0	99.78	2.79	95.66	64.92

Table 5: Release rates (%) with shared latent observations, $K = 32, n = 512$. Marginal score distributions are unchanged. Strong dependence can substantially change joint acceptance without invalidating the IUT argument.

B.2 Adaptive-selection stress test

Candidate j has independent two-point scores with mean -0.0205 and $n = 2048$; all tolerances are 0.02. Certification counts have dimensions $\text{trials} \times J$. Development counts are generated independently with the same distribution, and select the candidate with largest development mean, breaking ties by the first index. The fixed order is the numerical candidate index and is frozen

before any counts are observed. The fixed-sequence implementation takes cumulative products of pass indicators, so a failure can never be followed by another valid certification. All raw development and certification counts are saved.

B.3 Utility selection

For each of eight slices, the two Rademacher coordinate means in (15) are represented by independent Binomial($n, 1/2$) counts. The same per-slice observations determine all candidate scores within a split, preserving their strong candidate dependence. Independent streams generate the development and certification splits.

Let $r_n(\lambda) = 0.4\|\lambda\|_1\sqrt{\log(1/\alpha)/(2n)}$. Development search solves the finite problem

$$\max_{\lambda \in \mathcal{G}} \hat{U}_{\text{dev}}(\lambda) \quad \text{subject to} \quad \hat{D}_{s,\text{dev}}(\lambda) - r_n(\lambda) - \frac{0.12\|\lambda\|_2}{\sqrt{n}} \geq -0.01 \quad \forall s. \quad (21)$$

The last term is a fixed anticipatory design buffer, not a claimed optimal acquisition rule. Certification uses only r_n and fresh observations. The incumbent is explicitly included. All families share the same trials, scores, tolerances, buffer, and gate; only the candidate grid changes. The 21-point scalar grid has coordinates $j/20$, the 441-point scalar grid has coordinates $j/440$, and the block grid is their 21×21 coarse Cartesian product. Search is exhaustive on these small grids; no neural layer localization or coordinate-ascent convergence is inferred.

B.4 Pairing diagnostic

To isolate variance, define independent Rademacher variables U, V and scores

$$h(\theta_0, Z) = 0.5 + 0.20U, \quad h(\theta, Z) = 0.5 + 0.20U + 0.01 + 0.02V.$$

Both scores lie in $[0, 1]$. A paired mean difference from $n = 256$ observations has variance $0.02^2/n$. Independent incumbent and candidate examples instead give variance $(2 \cdot 0.20^2 + 0.02^2)/n$, a factor of 201. The executed simulation gives an empirical factor of 200.2. This diagnostic verifies the benefit under positive shared covariance, not universal superiority of pairing for every data-generating process.

B.5 Saved files and statistical uncertainty

The `base_upgrade.py` script writes calibration, correlation, adaptive-selection, utility, and variance CSV files. Compressed NPZ files store the original sufficient statistics needed to recompute the gates. The utility trial file records each proposed and released coefficient pair, certification outcome, utility, and true violation indicator. `manifest.json` records software versions, the root seed, settings, and hashes of generated results; figure PDFs and PNGs are generated by the same script.

For a count c of events among R independent trials, the reported two-sided 95% interval is

$$\left[\text{Beta}^{-1}(0.025; c, R - c + 1), \text{Beta}^{-1}(0.975; c + 1, R - c) \right],$$

with endpoints 0 or 1 at the corresponding extremes. Utility intervals use mean ± 1.96 Monte Carlo standard errors. These intervals describe repeated simulation, not uncertainty in a model's population score within one trial. Random data uncertainty and Monte Carlo uncertainty are different layers.

C Public digits checkpoint study

C.1 Frozen training and evaluation protocol

The data contain 1,797 examples, 64 features, and ten digit classes. Features are divided by 16. Stratified splits allocate 718 examples to training, 359 to development, 360 to certification, and 360 to reporting. The five seeds are 1701–1705. A 64–48–10 network uses a ReLU hidden layer, mini-batch size 128, learning rate 0.15, and momentum 0.9. Incumbent training runs 12 epochs, followed by 24 continuation epochs on classes 0–4. The update shares exact parameter ancestry with the incumbent.

The fixed development objective assigns each class 0–4 weight 0.16 and each class 5–9 weight 0.04. Each slice is a class and uses accuracy. Scalar interpolation evaluates 121 coefficients in $[0, 1]$; layer interpolation evaluates an 11×11 grid, grouping each affine layer’s weight and bias together. Both use the development constraint $\hat{D}_s \geq -0.03$. The proposal policy then freezes a candidate using development results before applying a single all-class Hoeffding gate with $\alpha = 0.05$, $\varepsilon_s = 0.03$, and paired accuracy differences in $[-1, 1]$. Reporting results do not alter training, search, or the gate.

The dataset source and content hash are recorded; features are reloaded from the documented scikit-learn dataset. All split indices, trained checkpoints, model predictions, per-class counts, proposal coefficients, and pass/fail decisions are saved. A finite-difference check of the implemented network gradient gave maximum error below 4.1×10^{-11} . The simulation seed uncertainty is not reused as a formal population confidence interval.

Model	Reporting accuracy	Weighted reporting accuracy
Incumbent	0.9478 ± 0.0077	0.9479 ± 0.0098
Continued checkpoint	0.7300 ± 0.0202	0.8813 ± 0.0113
Scalar proposal	0.9494 ± 0.0087	0.9544 ± 0.0123
Layer proposal	0.9483 ± 0.0089	0.9546 ± 0.0115
Actually released	0.9478 ± 0.0077	0.9479 ± 0.0098

Table 6: Mean $\pm$ descriptive sample standard deviation across five fixed split/training seeds. These are not five independent population confidence intervals. All five release attempts return the incumbent; the proposal rows are descriptive and are not deployed results.

C.2 Why fallback is an informative result

For a class with $n_s = 36$, the generic paired Hoeffding radius is $\sqrt{2 \log(20)/36} \approx 0.408$, far exceeding the 0.03 tolerance. The required empirical improvement is approximately 0.378; for an incumbent class accuracy above 0.9, even a perfect candidate cannot achieve that improvement on the same examples. A candidate that agrees almost perfectly with a strong incumbent may have a near-zero observed difference yet fail this gate. For a hypothetical true difference of zero, merely making the radius smaller than 0.03 would require approximately $2 \log(20)/0.03^2 \approx 6658$ iid examples per slice. This is a diagnostic calculation, not a sufficient condition for high joint power or a recommended fixed sample size.

These bounds can be conservative, and specialized paired classification tests or valid empirical-variance bounds may help. We did not revise the gate after observing these failures. This test primarily checks the data flow, common checkpoint ancestry, matched search counts, and exact fallback; the synthetic study is where population release probabilities are known and repeatedly

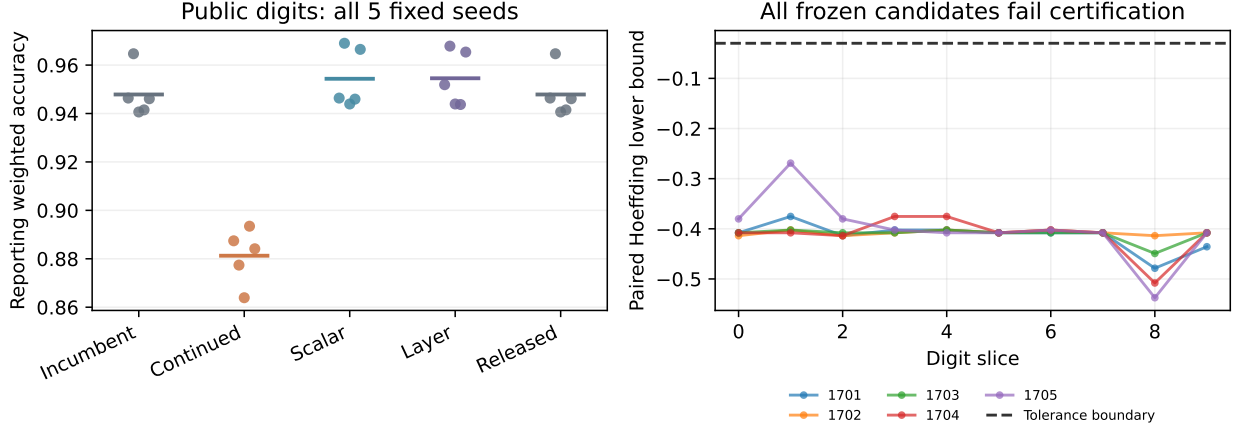


Figure 4: Completed public-data integration study. The proposed merges improve substantially over the deliberately impaired continuation, but the release gate does not certify the frozen proposal in any run. Small per-class certification samples make fallback an expected and informative outcome.

assessed. The public digits sample is also not a translation model, a modern base-model upgrade, or evidence of layer-specific capability localization.

C.3 Subsequent balanced continuation: a descriptive extension

The selective continuation above reduces aggregate reporting accuracy, which limits its resemblance to a useful base-model upgrade. After reviewing that result, we specified a second protocol before executing it: keep the same seeds, splits, incumbent training, optimizer, objective weights, tolerance, search counts, tie rules, and gate, but continue on all 718 training examples. The implementation retrains each incumbent deterministically and checks its tensors and split indices against the original archive; all five reproduce exactly. The continuation optimizer starts with zero momentum buffers in both paths. Balanced continuation takes 144 mini-batch updates over 24 epochs, versus 72 in the selective path. The two paths are therefore not a compute-matched intervention on class coverage.

Table 7 reports every policy from both paths. Balanced continuation improves mean overall and weighted reporting accuracy, yet both merge proposals have lower mean accuracy than the fully continued checkpoint. The layer proposal also has lower mean accuracy than the scalar proposal. These outcomes do not support layer superiority in this example. Mean accuracy improvement does not imply every seed or every class improves: Table 8 retains each seed and the chosen proposal’s worst observed class difference. These are finite-reporting-set differences, not population certificates.

The balanced protocol and source/helper hashes were saved before its training began; original split indices, checkpoints, all development candidates, predictions, losses, and gate values are archived in `results/digits_balanced/`. A separate verification script reconstructs reporting metrics and certification lower bounds from saved predictions for all ten seed/path runs. It also regenerates these tables. The original selective results are retained without modification.

This extension is subsequent and descriptive. Its certification and reporting partitions overlap the already examined selective study exactly within each seed, and the five seeds reuse one dataset. We do not treat the new outcomes as fresh confirmatory release certification, combine the two studies to claim a global error probability, or interpret seed variability as independent population uncertainty. The gate itself was executed unchanged, and no outcome triggered tuning. A future confirmatory deployment would require independent new data or a valid sequential design. Every

Path	Model	Accuracy	Weighted accuracy	Worst slice Δ
Selective	Incumbent	0.9478 ± 0.0077	0.9479 ± 0.0098	0.0000 ± 0.0000
	Continued checkpoint	0.7300 ± 0.0202	0.8813 ± 0.0113	-0.9271 ± 0.0464
	Scalar proposal	0.9494 ± 0.0087	0.9544 ± 0.0123	-0.0286 ± 0.0495
	Layer proposal	0.9483 ± 0.0089	0.9546 ± 0.0115	-0.0456 ± 0.0432
	Released	0.9478 ± 0.0077	0.9479 ± 0.0098	0.0000 ± 0.0000
Balanced	Incumbent	0.9478 ± 0.0077	0.9479 ± 0.0098	0.0000 ± 0.0000
	Continued checkpoint	0.9594 ± 0.0107	0.9637 ± 0.0105	-0.0114 ± 0.0156
	Scalar proposal	0.9567 ± 0.0109	0.9587 ± 0.0118	-0.0113 ± 0.0154
	Layer proposal	0.9544 ± 0.0116	0.9558 ± 0.0122	-0.0113 ± 0.0154
	Released	0.9478 ± 0.0077	0.9479 ± 0.0098	0.0000 ± 0.0000

Table 7: Both completed digits paths, mean $\pm$ descriptive seed SD. “Worst slice Δ ” is the per-seed minimum reporting class-accuracy difference from its incumbent, summarized over seeds. All frozen proposals fail; every released model is the incumbent. Balanced continuation was specified after the selective study and reuses its evaluation partitions.

Path/seed	Old	Updated	Scalar	Layer	Chosen (λ_1, λ_2)	Min. Δ	Gate
S/1701	0.9500	0.7361	0.9500	0.9556	L (0.100, 0.000)	0.0000	Fail
S/1702	0.9528	0.7139	0.9556	0.9528	S (0.025, 0.025)	0.0000	Fail
S/1703	0.9361	0.7111	0.9361	0.9361	S (0.067, 0.067)	-0.0286	Fail
S/1704	0.9444	0.7611	0.9472	0.9417	L (0.100, 0.000)	-0.0556	Fail
S/1705	0.9556	0.7278	0.9583	0.9556	S (0.117, 0.117)	-0.1143	Fail
B/1701	0.9500	0.9611	0.9583	0.9472	L (0.000, 1.000)	-0.0286	Fail
B/1702	0.9528	0.9667	0.9667	0.9667	S (0.917, 0.917)	0.0000	Fail
B/1703	0.9361	0.9472	0.9417	0.9417	S (0.592, 0.592)	0.0000	Fail
B/1704	0.9444	0.9500	0.9500	0.9500	L (0.400, 0.800)	-0.0278	Fail
B/1705	0.9556	0.9722	0.9667	0.9667	L (0.900, 0.400)	0.0000	Fail

Table 8: All ten seed/path outcomes. S and B denote selective and balanced continuation; the four accuracy columns use the reporting split. The chosen proposal is fixed using development observations; S/L in its column denotes scalar/layer interpolation. Coefficients are displayed to three decimals and saved at full precision. “Min. Δ ” is the chosen proposal’s worst reporting class difference. Gate outcomes use the separate within-run certification split; no failed result was omitted.

balanced proposal also fails the conservative gate, so the additional path establishes neither a successfully certified public-model improvement nor a useful empirical merging advantage.

D A concrete next-stage foundation-model protocol

This appendix specifies prospective work and contains no completed foundation-model result. First, choose at least two public, same-ancestry checkpoint transitions and verify their tensor mappings, token indices, licenses, and permitted redistribution. A version name or shared architecture is not sufficient. Include every prespecified pair in reporting; any path-quality exclusion must use development data and be disclosed.

Second, choose fixed language/domain slices and a bounded, interpretable per-example primary score. If corpus BLEU or a learned metric is also reported, use the appropriate paired document/source-cluster resampling and separate approximate inference from the finite-sample primary certificate. Multilingual translations of the same source should remain in the same split

and resampling cluster. Set tolerances using application meaning, not the observed uncertainty of the candidate one wishes to release.

Third, compare incumbent, full update, scalar merge, block merge, a Pareto-front generator, and a repair/replay candidate under declared evaluation and compute budgets. Preserve the same incumbent and certification interface for every method. Development may tune methods within the budget; the certification split cannot serve as an iterative optimization oracle. If several methods can produce the finally released model, either select the method on development data or apply candidate-level multiplicity control.

Finally, record generation tokens, wall-clock evaluation cost, memory, training cost, split identifiers, seeds, candidate counts, gate outcomes, and independent reporting scores. Publish failed gates, negative results, and incompatible checkpoint pairs. A useful empirical claim would require accepted non-incumbent utility gains over matched baselines across these transitions, not only an attractive development frontier or an inability to detect harm.